

Dual-MCU IoT Sensor Node

Complete Engineering Master Plan — Final Edition

STM32F411 Black Pill + ESP32 DevKit V1 · Fedora Linux · 8–10 Months

Register-Level Drivers · FreeRTOS · Custom Bootloader · OTA via MQTT · KiCad PCB

Includes: System Architecture · ADRs · Traceability Matrix · Risk Register · Quantitative Exit Gates · Debugging Philosophy

Your Hardware

MCUs	STM32F411CEU6 (Black Pill) · ESP32 DevKit V1
Debug	ST-Link V2 Clone (verified working)
Sensors	DHT22 · BMP280 · HC-SR04 · SSD1306 OLED 0.96"
Passives	10kΩ ×10 · 4.7kΩ ×10 · 1kΩ ×10 · 100nF ×2 · 10μF ×2 · RGB LEDs ×6
Misc	8-ch Logic Level Shifter · Multimeter · Soldering iron
Host OS	Fedora Linux (Hypriand DE)

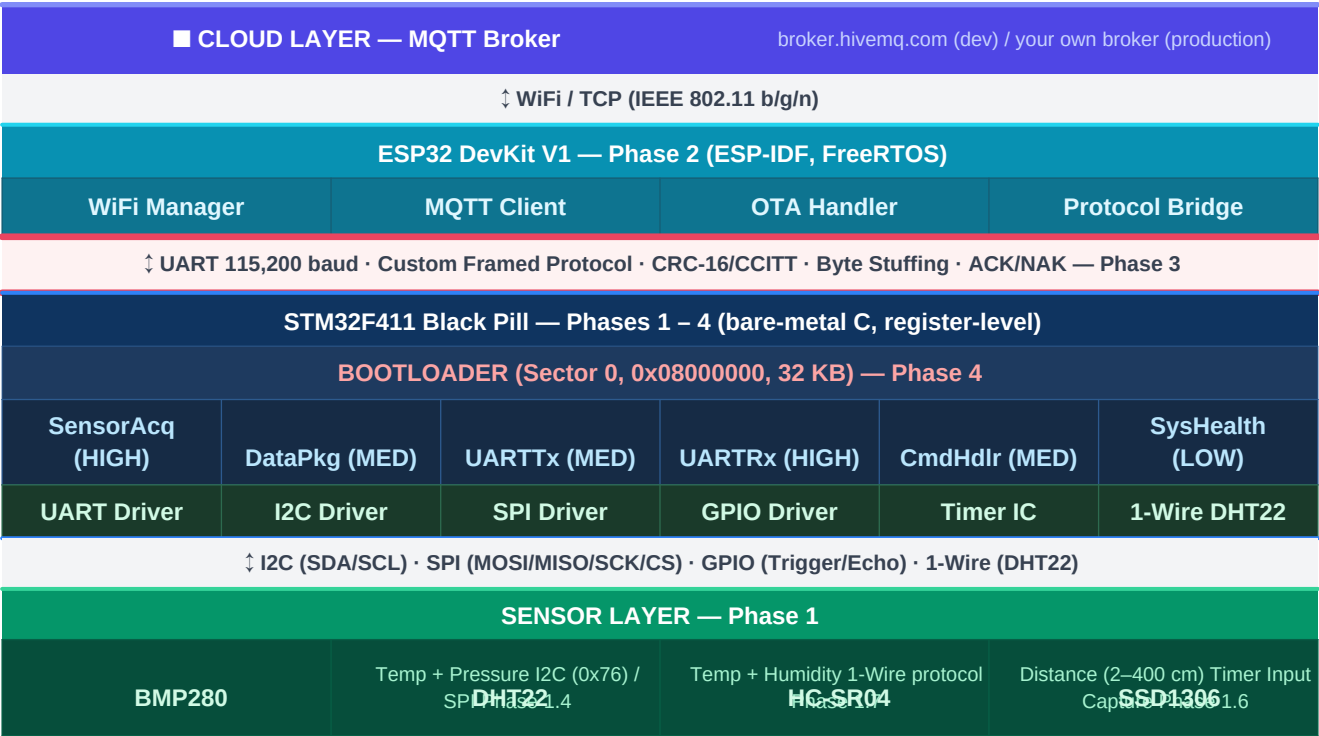
Plan Architecture

This plan is built around a dependency graph, not a calendar. Each phase has quantitative exit gates — measurable criteria that must be met before advancing. Time estimates should be recalibrated after Phase 0.

Phase	Name	Core Deliverable	Est.
0	Foundation	Toolchain live, ARM mental model solid, repo created	~3 wks
1	Peripheral Drivers	UART, I2C, SPI, Timers — register-level, all tested	~7 wks
2	FreeRTOS + ESP32	RTOS task architecture + ESP32 WiFi/MQTT baseline	~6 wks
3	Inter-MCU Protocol	Framed UART protocol with CRC, validated	~5 wks
4	Bootloader + OTA	Custom bootloader + end-to-end OTA pipeline	~8 wks
5	Integration	72-hour soak test, watchdog, error recovery	~4 wks
6	PCB	KiCad design + fabricated board running firmware	~6 wks

System Architecture

Every component in this diagram is something you build and own. There are no black boxes. The colour of each layer matches the phase in which it is built.



Data flow:
BMP280/DHT22/HC-SR04

→ SensorAcq task → SensorDataQueue → DataPkg task → TxQueue → UARTTx → UART wire → ESP32 Protocol Bridge → MQTT → Cloud

Traceability Matrix

Every sensor reading has a complete path from hardware to cloud. If any cell in a row is blank, that path is incomplete. Use this table to verify architectural coherence at any phase.

Sensor	Interface	Driver (Phase 1)	RTOS Task (Phase 2)	Queue	Transport	MQTT Topic
BMP280	I2C (0x76)	drivers/bmp280	SensorAcquisition	SensorDataQ	UART→ESP32→WiFi	device/001/sensors / temperature pressure
DHT22	1-Wire	drivers/dht22	SensorAcquisition	SensorDataQ	UART→ESP32→WiFi	device/001/sensors / humidity
HC-SR04	Timer IC	drivers/hcsr04	SensorAcquisition	SensorDataQ	UART→ESP32→WiFi	device/001/sensors / distance
SSD1306	I2C (0x3C)	drivers/ssd1306	SensorAcquisition (display only)	— local display no queue needed	Local GPIO only	— no cloud path (display)

Architectural Decision Records

ADRs document the *why* behind major design decisions. When you are tempted to switch frameworks or change a bus mid-project, read the relevant ADR first. The consequences column tells you what breaks.

ADR-01 UART for Inter-MCU Communication (not SPI)	
Context:	Two MCUs must exchange sensor data, config, and OTA firmware. Both SPI and UART are viable. SPI can run at 10+ Mbps, UART tops out at ~1 Mbps.
Decision:	Use UART with a custom framed protocol (SOF/EOF/CRC-16) at 115,200 baud.
Bandwidth:	14.4 KB/s — sufficient for sensor data (100 B/s) and OTA (87 KB in ~8s).
Wiring:	2 wires + GND vs 4–5 wires for SPI. Simpler PCB routing in Phase 6.
Debugging:	UART frames are human-readable in a terminal. SPI requires a logic analyser.
Consequence:	OTA takes ~15 seconds. Acceptable for an infrequent operation.

ADR-02 ESP-IDF Framework (not Arduino for ESP32)	
Context:	ESP32 can use the Arduino framework (familiar) or native ESP-IDF (more complex). Both support WiFi and MQTT.
Decision:	Use ESP-IDF exclusively throughout the project.
OTA:	Phase 4 OTA requires direct access to esp_ota_begin/write/end APIs. Arduino abstracts these — custom OTA flow requires either bypassing the abstraction or reimplementing it.
MQTT:	ESP-IDF's esp-mqtt client handles reconnection and QoS more robustly than Arduino's PubSubClient library.
FreeRTOS:	Educational goal: understand FreeRTOS on both MCUs. Arduino hides FreeRTOS behind loop() and callbacks.
Consequence:	Phase 2 ESP32 setup is more involved. The payoff is full control over the OTA partition table in Phase 4.

ADR-03 Register-Level Drivers (not STM32 HAL)	
Context:	STM32 can be programmed via the HAL library (high-level, ST-maintained) or by direct register access (low-level, manual). HAL is faster to get running; register-level is harder but more transparent.
Decision:	Write all STM32 drivers at register level. No HAL, no CubeMX code generation.
Education:	Primary goal: understand how peripherals actually work at hardware level. HAL hides every register access behind a function call.
Bootloader:	Custom bootloader must fit in 32 KB (Sectors 0–1). HAL alone pulls in 40–60 KB. Register-level bootloader = 14 KB.
Debugging:	When a peripheral fails, you read RM0383 directly. With HAL you hunt through 10,000 lines of abstraction.
Consequence:	Phases 0–1 are harder. Phase 4 bootloader is tractable. Every line of firmware is owned and understood.

Debugging Philosophy & Instrumentation Strategy

This section is a reference, not a tutorial. Bookmark it. When something breaks — and it will — come here first before spending hours on random changes. Every tool listed is available on your Fedora setup.

Principle 1 — UART printf is your primary visibility window (Phases 0–3)

For the first four months, UART printf is your oscilloscope. Every driver ships with a `driver_dump_regs()` function. Every critical section has a diagnostic printf before you commit. Add debug levels so you can turn verbosity up and down without removing statements:

```
#define DBG_ERROR 1 // always on
#define DBG_INFO 2 // default during development
#define DBG_TRACE 3 // single function only – very verbose
#define DEBUG_LEVEL DBG_INFO

#if DEBUG_LEVEL >= DBG_INFO
uart_printf("[I2C] CCR=0x%04X TRISE=%d\r\n", I2C1->CCR, I2C1->TRISE);
#endif
```

Principle 2 — HardFault handler is non-negotiable (implement in Phase 1, test immediately)

Without this you get a silent lockup with zero information when code crashes. With it you get the exact instruction address printed over UART. Implement it the day UART TX works. Test it intentionally:

```
void HardFault_Handler(void) {
volatile uint32_t *sp = (volatile uint32_t *)__get_MSP();
uart_printf("** HARDFAULT ** PC=0x%08lX LR=0x%08lX\r\n",
sp[6], sp[5]);
while(1);
}

// Decode address to source line:
// arm-none-eabi-addr2line -e firmware.elf 0x0800XXXX
```

Principle 3 — Logic Analyser — buy before Phase 3

A Cypress FX2-based clone (~\$1200 from AliExpress) with Sigrok/PulseView (free, sigrok.org) decodes UART frames, I2C transactions, and SPI timing visually. Protocol debugging without seeing actual bytes on the wire is guesswork. Install Sigrok on Fedora:

```
sudo dnf install sigrok pulseview

# Plug in FX2 analyser, run: pulseview

# Add UART decoder: protocol decoders → UART → set baud to 115200

# Trigger on 0xAA (your SOF byte) to capture complete frames
```

Principle 4 — GDB + OpenOCD — for Phase 4 bootloader development

When printf is not enough (register-level flash operations, jump sequence), use GDB. OpenOCD bridges between GDB on your PC and the STM32 over SWD:

```
# Terminal 1: start OpenOCD server
```

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg

# Terminal 2: connect GDB
arm-none-eabi-gdb firmware.elf
(gdb) target remote :3333
(gdb) break bootloader_jump_to_app
(gdb) continue
(gdb) print/x SCB->VTOR # inspect vector table register
(gdb) print/x *(uint32_t*)0x08010000 # read first word of app flash
(gdb) stepi # single-step one assembly instruction
```

Principle 5 — Systematic Root Cause Analysis — the five-step process

Random changes waste hours. This process finds the root cause faster:

Step	Action	Why
1. Reproduce	Make the failure happen consistently. If you can't reproduce it, it's a race condition.	An intermittent bug cannot be fixed without a deterministic reproduction.
2. Binary search	Add printf at function entry, then mid-function, narrow to the failing line.	Fastest way to isolate in a system without a debugger attached.
3. Dump registers	Call driver_dump_regs() and compare every value against RM0383.	90% of bare-metal bugs are wrong register values — missing clock enable, wrong mode bit, wrong BRR.
4. Read the RM	Find the register description in RM0383. Read the whole section, not just the field name.	The errata and notes sections contain the exact bug you're hitting.
5. Time-box	If unresolved after the circuit-breaker time in the risk register, pivot. Log as technical debt.	Spending 3 weeks on one bug kills the project. The pivot is not failure.

PHASE 0 Foundation

~3 weeks

Entry Gate Intermediate C, basic Arduino experience.

Exit Gate Can compile + flash a bare-metal project, ARM memory map understood, repo live.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
Makefile build fails with cryptic gcc error	High	2 days	Use pre-made Makefile from stm32world.com, study it line by line, then write your own. Focus on understanding flags, not fighting build system.
OpenOCD cannot find ST-Link (permission error)	High	1 day	Run 'lsusb grep STMicro'. If listed, udev rule is missing. If not listed, USB cable is data-only (power-only cables are common). Try a different cable.
Linker script errors — undefined symbols, wrong addresses	Medium	2 days	Use cpq/bare-metal-programming-guide linker script as reference. Compare your .map file addresses against RM0383 Section 2.3.

■ Phase Journey — How This Phase Fits Together

Phase 0 builds the floor before any walls go up. 0.1 hardens your C so register manipulation in Phase 1 does not surprise you. 0.2 replaces the Arduino IDE with a real toolchain — you write the Makefile, understand every flag, flash via OpenOCD. 0.3 puts the primary source documents in your hands. 0.4 creates the engineering record that runs alongside the entire project.

Dependency chain: 0.1 C Hardening → 0.2 Toolchain + Makefile → 0.3 Datasheets → 0.4 Repo

0.1 — C Language Hardening

Arduino hides most of this. Bare-metal exposes all of it. Close these gaps on paper and in small PC-side programs before touching the MCU.

- **Bitwise operations:** |= to set, &= ~mask to clear, ^= to toggle. Practice on paper: given reg=0x00000000, set bit 4, clear bit 2, toggle bit 4.
- **volatile keyword:** Tells the compiler this address can change outside program flow. Every peripheral register pointer must use volatile — without it, optimised builds silently drop writes.
- **Struct pointer casting:** ((USART_TypeDef *)0x40011000)->BRR = 0x683. You are casting a raw integer address to a struct pointer and accessing a field. When this clicks, the entire register-level API makes sense.
- **Function pointers:** The interrupt vector table is an array of function pointers. You need to understand the syntax to write interrupt handlers correctly.
- **const, static, inline, __attribute__:** Know what each does to the generated binary.

■ Before You Start This Section

C Pointers and Memory: If struct pointer casting feels shaky, spend 2–3 hours here before anything else.

beej.us/guide/bgc/ – Beej's Guide to C Programming, Chapters 7–8 (Pointers)

0.2 — Toolchain Setup on Fedora

■ What is a Makefile?

What it is: An IDE secretly runs a compiler and linker for you. A Makefile is those exact commands written explicitly as text. When you type 'make', it reads and executes them in order.

Why it's here: STM32 bare-metal requires flags (-mcpu=cortex-m4, -mfloat-abi=hard) that Arduino never exposes. Writing your own Makefile means you control every flag, understand every step, and can debug build failures.

Learn it: Makefile Tutorial
makefiletutorial.com

■ What is ST-Link, SWD, and OpenOCD?

What it is: Your STM32 has a debug port (SWD — Serial Wire Debug). The ST-Link V2 clone is a USB adapter that speaks USB on your PC side and SWD on the MCU side. OpenOCD drives the ST-Link. Chain: terminal → OpenOCD → ST-Link → SWD pins → STM32 flash controller.

Why it's here: The Black Pill has no built-in ST-Link. Without the udev rule, Linux refuses to open the USB device unless you run as root — which you must never do.

Learn it: OpenOCD documentation
openocd.org/doc/html/index.html

Install all tools:

```
sudo dnf install arm-none-eabi-gcc-cs arm-none-eabi-newlib arm-none-eabi-gdb openocd make git
```

ST-Link udev rule (do this before your first flash):

```
# /etc/udev/rules.d/70-stlink.rules
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3748", TAG+="uaccess"
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374b", TAG+="uaccess"
sudo udevadm control --reload-rules && sudo udevadm trigger
# Verify: lsusb | grep STMicro
```

Makefile flag reference:

Flag	What it does
-mcpu=cortex-m4	Generates Cortex-M4 instructions. Wrong value = binary will not run.
-mthumb	Use Thumb-2 instruction set. Mandatory for all Cortex-M code.
-mfpu=fpv4-sp-d16	Enable hardware FPU. Without it, float ops use slow software emulation.
-mfloat-abi=hard	Pass float args in FPU registers. Must match newlib-nano ABI.
-O0 -g3	No optimisation, full debug symbols. Use during development.
-Wl,--gc-sections	Remove unused code. Works with -ffunction-sections -fdata-sections.
0x08000000	STM32 flash start address. OpenOCD writes binary here on flash command.

0.3 — ARM Architecture Mental Model

■ What is the ARM Memory Map?

What it is: Every byte of the STM32's address space has a fixed address — flash, RAM, and every peripheral register. GPIOA lives at 0x40020000. USART1 at 0x40011000. When you write ((USART_TypeDef*)0x40011000)->BRR = 0x683 you are reaching into physical memory at that address and writing a 32-bit value.

Why it's here: This mental model is the foundation of everything in Phase 1. RM0383 Section 2.3 has the full memory map. Read it before Phase 1.

Learn it: RM0383 Section 2.3

[st.com](#) → [search RM0383](#), [download free PDF](#)

Documents to download now:

Document	Source	What you need it for
RM0383 — STM32F411 Reference Manual	st.com → search RM0383	Clock tree, peripheral registers, memory map. Open constantly in Phase 1.
STM32F411 Datasheet	st.com → search STM32F411CEU6	Pin definitions, alternate function table. Needed for Phase 1 + PCB.
ARM Cortex-M4 User Guide (DUI-0553)	developer.arm.com/documentation/dui0553	NVIC, SysTick, HardFault, exception types.
ESP32 Technical Reference Manual	espressif.com → Support → Documents	WiFi architecture, GPIO, UART. Needed from Phase 2 onward.

0.4 — Repository and Project Skeleton

```
dual-mcu-iot-node/
├── stm32/
│   ├── drivers/ ← Phase 1: one subdir per peripheral
│   ├── rtos/ ← Phase 2: FreeRTOS submodule + tasks
│   ├── bootloader/ ← Phase 4: separate binary, own Makefile
│   ├── app/ ← Main application
│   ├── startup/ ← startup_stm32f411ceux.s
│   └── esp32/main/ ← ESP-IDF application
├── protocol/
│   └── protocol_spec.md ← Single source of truth for both MCUs
├── docs/
│   ├── build.md ← Clone → build → flash in under 5 minutes
│   └── README.md
```

■ Resources for this Phase

bare-metal-programming-guide (cpq)

github.com/cpq/bare-metal-programming-guide

No-HAL, pure GCC + Make STM32 reference. Read alongside RM0383.

Makefile Tutorial

makefiletutorial.com

30-minute read. Enough to write and understand your Makefile.

Beej's Guide to C

beej.us/guide/bgc/

Free. Chapters 7–8 are the prerequisite for Phase 1.

Quantitative Exit Gate — Phase 0

All criteria below must be met before advancing to the next phase.

- `arm-none-eabi-gcc --version` returns a version string (toolchain confirmed installed)
- `lsusb | grep STMicro` shows the ST-Link device (udev rule working)
- `make` builds `firmware.elf` and `firmware.bin` without errors from your hand-written Makefile
- `openocd` flashes `firmware.bin` and reports 'verified' — LED blinks at 1 Hz
- GitHub repository is live with directory skeleton and `v0.0-toolchain-verified` tag
- You can explain every flag in your `CFLAGS` and `LDFLAGS` without looking them up

Tag: `v0.0-toolchain-verified` · YouTube: *"STM32 Dev Environment on Fedora — ARM GCC, OpenOCD, Makefile"*

PHASE 1 STM32 Bare-Metal Peripheral Drivers

~7 weeks

Entry Gate Phase 0 complete. Makefile works, OpenOCD flashes, RM0383 downloaded.

Exit Gate UART (ISR ring buffer + printf), GPIO, I2C (BMP280+SSD1306), SPI, Timer IC (HC-SR04), DHT22 — all with register-level test harnesses. HardFault handler working.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
STM32F4 I2C bus lockup after NACK (hardware errata)	High	5 days	Implement bit-bang I2C recovery routine (9 SCL pulses via GPIO). If hardware I2C remains unstable, implement fully bit-banged I2C driver. Log as technical debt.
UART RX ISR misses bytes at high baud rates	Medium	3 days	Reduce baud to 9600 for diagnosis. Check for missing RXNE check in ISR. Verify ring buffer overflow detection. Increase buffer size to 512 bytes.
HardFault with no printf output (HardFault handler not working)	Medium	1 day	Verify <code>uart_init()</code> is called before the fault. Check <code>HardFault_Handler</code> is in the vector table. Use GDB to inspect stack frame manually.
BMP280 WHO_AM_I returns 0xFF (no I2C response)	High	3 days	Systematic I2C fault diagnosis: check pull-ups (4.7kΩ to 3.3V), RCC clock enable, GPIO AF configuration, CCR calculation.

■ Phase Journey — How This Phase Fits Together

Phase 1 has one governing rule: UART first, everything else after. UART is your only visibility into the system until you have a logic analyser. Every other driver you write will use UART to show you what went wrong.

Dependency chain: GPIO+RCC → UART (debug window) → SysTick → I2C+BMP280+SSD1306 → SPI → Timer IC+HC-SR04 → 1-Wire+DHT22

Universal Driver Pattern (apply to every peripheral):

1. Read the full RM0383 chapter for the peripheral before writing a line of code.
2. Write init with every register write commented with its RM section and field name.
3. Write `driver_dump_regs()` — prints every relevant register value over UART.
4. Write `driver_test.c` — exercises every function, prints PASS/FAIL over UART.
5. Only commit when the test harness passes cleanly.

1.1 — GPIO + RCC

■ What is RCC — Why You Must Enable the Clock First

What it is: RCC (Reset and Clock Control) gates power to every peripheral. By default, all peripherals are unpowered. Before writing to any USART, I2C, SPI, or GPIO register, enable its clock.

Why it's here: The #1 silent bug for beginners: write to USART1->CR1 without enabling the USART1 clock. Nothing happens. No error. Just silence. The write does not stick because the peripheral has no power.

Enable clocks first. Configure MODER, OTYPER, OSPEEDR, PUPDR. Use BSRR (not ODR) for atomic set/reset. Test with RGB LEDs cycling all combinations.

1.2 — UART Driver (build this before any other driver)

■ What is a Circular Ring Buffer?

What it is: A fixed-size array where write and read pointers chase each other in a circle. The ISR writes bytes at the write pointer; your task reads at the read pointer. Power-of-2 size allows wrap via bitmask instead of modulo.

Why it's here: Your UART ISR fires on every received byte. A ring buffer lets the ISR drop bytes in at full speed while your task reads them at its own pace — no blocking, no missed bytes.

■ What is printf Retargeting?

What it is: printf() writes to stdout, which doesn't exist on bare metal. Overriding the _write() syscall redirects all printf output to your UART — a 5-line change that gives you formatted debug output for the entire project.

Why it's here: After this, uart_printf("Temp: %d.%02dC\r\n", t_int, t_frac) sends formatted sensor data over UART. This is your primary debug tool for the next four months.

Build in this order: polling TX → printf retargeting → ISR RX ring buffer → polled RX with timeout → HardFault handler.

HardFault handler — implement immediately after UART TX:

```
void HardFault_Handler(void) {
    volatile uint32_t *sp = (volatile uint32_t *)__get_MSP();
    uart_printf("*** HARDFAULT ** PC=0x%08lX LR=0x%08lX\r\n", sp[6], sp[5]);
    while(1);
}

// Decode: arm-none-eabi-addr2line -e firmware.elf 0x0800XXXX
```

1.3 — I2C Driver + BMP280 + SSD1306

■ What is I2C at the Protocol Level?

What it is: Two-wire bus: SDA (data) and SCL (clock). Master generates clock, initiates all transactions. Each device has a 7-bit address. Transaction: START → address+R/W → ACK from device → data bytes → STOP. Device acknowledges each byte by pulling SDA low.

Why it's here: BMP280 (0x76/0x77) and SSD1306 (0x3C/0x3D) are both I2C. Writing the I2C driver correctly unlocks both sensors.

Learn it: I2C Bus Spec

nxp.com/docs/en/user-guide/UM10204.pdf

STM32F411 I2C errata: peripheral locks with SDA stuck LOW after a NACK. Recovery: 9 SCL clock pulses via GPIO bit-bang, manual STOP, then re-init. Add a timeout to every SR1/SR2 polling loop. Mandatory in production code.

BMP280 bring-up: read register 0xD0 first. Expect 0x60. If you get 0x60, I2C works. If 0xFF, check pull-ups (4.7kΩ on SDA+SCL to 3.3V).

1.4 — SPI Driver

Implement SPI master mode 0 (CPOL=0, CPHA=0). Software CS via GPIO. Test against BMP280 in SPI mode — read WHO_AM_I register (bit 7 = 1 for read). Expect 0x60. BRR for <10 MHz: SPI1 on APB2 (84 MHz), BR=3 →

5.25 MHz.

1.5 — Timer Input Capture + HC-SR04

■ What is Timer Input Capture?

What it is: Hardware feature: the timer automatically records its counter value the instant a GPIO pin changes state (rising or falling edge). No CPU involvement, immune to context switches. The ISR fires on the edge and reads the captured count.

Why it's here: HC-SR04 echo pulse width (10µs–38ms) encodes distance. Capture rising and falling edge timestamps. Difference in µs / 58 = distance in cm.

HC-SR04 echo pin swings to 5V. STM32F411 GPIO max input is 3.6V. The 8-channel logic level shifter on the Echo line is mandatory.

1.6 — DHT22 Single-Wire

Proprietary 1-wire protocol: '0' bit = 26–28µs HIGH pulse, '1' bit = 70µs HIGH. Use Timer Input Capture — busy-wait timing breaks under FreeRTOS context switches. Pull-up: 4.7kΩ on data line to 3.3V. Checksum = sum of 4 data bytes & 0xFF.

■ Resources for this Phase

embetronicx.com STM32 Tutorials

embetronicx.com/tutorials/microcontrollers/stm32/

Register-level GPIO, UART, I2C, SPI, Timer tutorials.

BMP280 Datasheet

bosch-sensortec.com

Compensation formula in Section 4.2.3. WHO_AM_I at 0xD0.

DHT22/AM2302 Datasheet

cdn.sparkfun.com/assets/d/7/9/5/b/DHT22.pdf

Timing diagram in Section 5.

Quantitative Exit Gate — Phase 1

All criteria below must be met before advancing to the next phase.

- UART 10,000-byte loopback test completes with zero errors at 115,200 baud
- HardFault handler fires and prints PC + LR over UART on intentional null-pointer dereference
- BMP280 WHO_AM_I register (0xD0) returns 0x60 — I2C bus confirmed working
- 100 consecutive BMP280 temperature reads: zero I2C errors, zero checksum failures
- SSD1306 displays live temperature string updating every second
- HC-SR04 measures a 30cm target within ±5% (reads 28–32cm) over 20 consecutive samples
- DHT22 humidity within ±5% of a reference reading; checksum passes on every read
- 12-hour continuous sensor polling (all 4 sensors) with zero HardFaults in UART log

Tag: v0.1-peripheral-drivers · YouTube: "STM32 UART From Scratch — No HAL" · "I2C Errata: The Bug That Took 3 Days"

PHASE 2 FreeRTOS + ESP32 Baseline

~6 weeks

Entry Gate Phase 1 complete. All peripheral drivers tested and committed.

Exit Gate STM32 running 6-task FreeRTOS with sensor data in queues. ESP32 publishing live sensor values to MQTT visible in MQTT Explorer.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
FreeRTOS stack overflow under full system load	High	3 days	Enable configCHECK_FOR_STACK_OVERFLOW=2. Double suspect task stacks. Use uxTaskGetStackHighWaterMark() to measure actual usage.
ESP-IDF build system errors on Fedora	Medium	2 days	Ensure Python venv is activated: '. \$IDF_PATH/export.sh'. Use idf.py instead of cmake directly. Check esp-idf version matches documentation.
MQTT broker connection refused or instant disconnect	Medium	2 days	Test with MQTT Explorer first (confirms broker is reachable). Check WiFi IP assignment before MQTT connect. Verify broker URL and port (1883 no TLS, 8883 TLS).
Priority inversion deadlock between FreeRTOS tasks	Medium	3 days	Replace binary semaphores protecting shared resources with mutexes. Enforce global resource acquisition order (I2C before UART, never reversed).

■ Phase Journey — How This Phase Fits Together

Two parallel workstreams from week 3. Workstream A (FreeRTOS on STM32) is harder — do it first. Workstream B (ESP32) can start once your first FreeRTOS task is running.

Dependency chain: A: Port FreeRTOS → A: Task architecture → A: Verify with UART → B: ESP-IDF setup → B: WiFi station → B: MQTT publish/subscribe

Workstream A — FreeRTOS on STM32

■ What is an RTOS?

What it is: Without an RTOS, your loop runs tasks sequentially. If reading a sensor takes 80ms and a command arrives during that time, you miss it. An RTOS runs multiple tasks that appear simultaneous by switching between them thousands of times per second.

Why it's here: Your sensor node has concurrent jobs: acquire sensors, send UART, receive commands, respond to OTA. Doing these in a single loop creates timing conflicts. FreeRTOS solves this cleanly.

Learn it: FreeRTOS Book (Richard Barry)

freertos.org/Documentation/RTOS_book.html

■ Tasks, Queues, Semaphores, Mutexes

What it is: Task: a function that runs forever in a loop. Queue: thread-safe FIFO pipe between tasks. Semaphore: signalling ('this event happened, wake up'). Mutex: lock with priority inheritance ('only one task uses this at a time').

Why it's here: Use a mutex (not binary semaphore) to protect shared resources — mutexes implement priority inheritance, preventing the priority inversion deadlock pattern.

Task Architecture:

Task	Priority	Stack (words)	Job
SensorAcquisition	HIGH	512	Reads BMP280, DHT22, HC-SR04 on schedule. Posts to SensorDataQueue.
DataPackager	MEDIUM	256	Consumes SensorDataQueue. Formats protocol frames. Posts to TxQueue.
UARTTransmit	MEDIUM	256	Consumes TxQueue. Sends frames to ESP32 over UART.
UARTReceive	HIGH	512	Drains ISR ring buffer byte by byte. Feeds protocol parser.
CommandHandler	MEDIUM	256	Processes complete frames from CmdQueue. Handles config + OTA triggers.
SystemHealth	LOW	128	Kicks watchdog. Logs heap and uptime to MQTT every 10 seconds.

Critical rule: Any FreeRTOS API call inside an ISR must end in `FromISR`. `xQueueSendFromISR()` not `xQueueSend()`. Calling the wrong version corrupts the scheduler and causes crashes that appear only after hours of operation.

Workstream B — ESP32 + ESP-IDF

■ What is MQTT?

What it is: Publish-subscribe protocol for IoT. Devices publish to topics; subscribers receive messages. The broker routes between publishers and subscribers. Unlike HTTP, MQTT maintains a persistent TCP connection with tiny overhead.

Why it's here: Your ESP32 publishes sensor data to `device/001/sensors/*` and subscribes to `device/001/ota/*` and `device/001/config/*`. One broker serves all topics.

Learn it: HiveMQ public broker (free, no account)

broker.hivemq.com port 1883

Do not use the Arduino framework for ESP32 in this project. Phase 4 OTA requires direct access to `esp_ota_begin/write/end` APIs which Arduino abstracts in ways that prevent the custom OTA flow.

■ Resources for this Phase

FreeRTOS Book

freertos.org/Documentation/RTOS_book.html

Free PDF. Read Chapters 1–5 before writing your first task.

ESP-IDF Get Started (Linux)

docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/linux-macos-setup.html

Official Fedora-compatible setup guide. Follow exactly.

MQTT Explorer

mqttexplorer.com

Free desktop MQTT client. Verify your ESP32 is publishing correctly.

Quantitative Exit Gate — Phase 2

All criteria below must be met before advancing to the next phase.

- All 6 FreeRTOS tasks shown as running in `vTaskGetRunTimeStats()` output
- Intentional stack overflow fires `vApplicationStackOverflowHook` with correct task name
- Sensor data visible flowing through all queues — confirmed via UART debug output
- No stack overflow in 24-hour continuous run under full sensor load
- ESP32 connects to WiFi and obtains IP address — confirmed in UART log
- MQTT publish visible in MQTT Explorer for at least 5 minutes of continuous publishing
- MQTT subscribe receives a test command sent from laptop

Tags: [v0.2-freertos-tasks](#) · [v0.2-esp32-mqtt-baseline](#)

PHASE 3 Inter-MCU UART Protocol

~5 weeks

Entry Gate	Phase 2 complete. FreeRTOS running. ESP32 publishing to MQTT.
Exit Gate	STM32 and ESP32 exchanging structured CRC-verified frames. Packet loss and corruption detected, NAK'd, and retransmitted.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
Protocol parser stuck in non-IDLE state (partial frame, no timeout)	High	2 days	Implement 500ms parser timeout immediately. Without it one bad frame hangs the entire communication channel permanently.
CRC mismatch on valid frames (implementation bug)	Medium	3 days	Verify CRC against crccalc.com with at least 5 known byte sequences. Check both STM32 and ESP32 use identical polynomial, init value, and byte order. Mismatch in any parameter gives a completely different CRC.
Byte stuffing decoder edge case corrupting frames	Medium	2 days	Test explicitly: payload containing 0xAA, 0x55, 0xBB back-to-back. Test payload of all 0xAA bytes (worst-case expansion). If complexity is too high, simplify to no-stuffing with a larger SOF marker.

■ Phase Journey — How This Phase Fits Together

Design-first. The first two weeks produce `protocol_spec.md` and a desktop-verified CRC implementation before a single line runs on an MCU. A protocol bug found on a PC takes 5 minutes to fix; the same bug on two MCUs takes hours.

Dependency chain: Write spec → CRC on PC (unit tested) → Byte stuffing → Parser state machine → Port to both MCUs → Logic analyser validation

■ What is a Framing Protocol?

What it is: When two devices exchange bytes over UART, the receiver needs to know where one message ends and the next begins — UART provides no boundaries. A framing protocol adds explicit SOF/EOF markers, a length field, a command byte, and a checksum.

Why it's here: Without framing, a missed byte shifts every subsequent byte by one, silently corrupting all data forever. Framing gives synchronisation (SOF re-locks after any error) and integrity (CRC detects flipped bits).

■ What is CRC?

What it is: CRC-16/CCITT: a 16-bit checksum computed over payload bytes using polynomial 0x1021, init 0xFFFF. Both sender and receiver compute it independently. Any single bit flip in transit produces a different CRC — the receiver detects the corruption.

Why it's here: Implement table-driven CRC on your PC first. Verify against crccalc.com with 5 known inputs before porting to the MCU. Both STM32 and ESP32 must use identical parameters.

Learn it: CRC Calculator (verify your implementation)

crccalc.com

Frame Format

Field	Size	Value	Purpose
SOF	1 byte	0xAA	Start-of-frame marker. Parser resets state machine here.
LEN	2 bytes	uint16 big-endian	Payload length in bytes (pre-stuffing).
SEQ	1 byte	0-255 rolling	Sequence number for ACK matching.
CMD	1 byte	See command table	Defines how receiver interprets the payload.
PAYLOAD	LEN bytes	Command-specific	Variable length. Zero bytes valid (e.g. PING).
CRC16	2 bytes	CRC-16/CCITT	Covers LEN+SEQ+CMD+PAYLOAD. Not SOF or EOF.
EOF	1 byte	0x55	End-of-frame confirmation.

Byte stuffing: 0xAA→{0xBB,0x8A}, 0x55→{0xBB,0x75}, 0xBB→{0xBB,0x9B}. Document in protocol_spec.md before implementing. Buy your logic analyser before this phase — Cypress FX2 clone (~\$1200) with Sigrok/PulseView (free, sigrok.org).

■ Resources for this Phase

Sigrok/PulseView

sigrok.org/wiki/Downloads

Free, open-source. Works with cheap FX2 USB analysers on Fedora. UART decoder built-in.

CRC Calculator

crccalc.com

Select CRC-16/CCITT. Compare against your C implementation output.

Quantitative Exit Gate — Phase 3

All criteria below must be met before advancing to the next phase.

- CRC-16/CCITT implementation matches crccalc.com for 5 different input byte sequences
- Byte stuffing round-trip: encode then decode returns original for 10 different payloads
- Parser state machine resets to IDLE after 500ms with no bytes (timeout confirmed via UART log)
- STM32 SENSOR_DATA frame visible decoded in Sigrok/PulseView logic analyser capture
- ESP32 decodes SENSOR_DATA, live sensor values appear in MQTT Explorer
- Injected 1-bit payload corruption: CRC detected, NAK sent by receiver, frame retransmitted
- PING from ESP32 receives PONG from STM32 within 100ms — 20 consecutive PING/PONG pairs

Tag: v0.3-inter-mcu-protocol · YouTube: "Designing a Binary UART Protocol From Scratch"

Entry Gate	Inter-MCU protocol passing all acceptance tests.
Exit Gate	Push firmware_v2.bin to MQTT from laptop. STM32 receives, flashes, verifies, reboots into new firmware. Zero physical access required.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
Wrong FLASH ORIGIN in application linker script (HardFault on jump)	High	2 days	Triple-check: application ORIGIN must be 0x08010000 not 0x08000000. Verify with GDB: after flash, read (uint32_t*)0x08010000 — must be a valid stack pointer.
OTA partially written before power loss (device bricks)	High	3 days	Write OTA 'valid' flag LAST, after full CRC verification. Bootloader must check flag before jumping — corrupt flag = wait for new OTA.
Bootloader too large for 32KB (sectors 0+1)	Medium	2 days	Run 'make size' on bootloader binary immediately. Remove all printf from release build (-DNDEBUG). If still over 32KB, move bootloader to sectors 0+1+2 (48KB) and adjust app ORIGIN.

■ Phase Journey — How This Phase Fits Together

Two sequential tracks. First: build the bootloader (flash layout, linker scripts, app jump). Then: build the OTA receive chain on ESP32. Only when both are complete can you do the end-to-end test — laptop MQTT push → automatic firmware update.

Dependency chain: Flash layout → Two linker scripts → Bootloader binary → Flash write driver → App jump sequence → ESP32 OTA receiver → End-to-end test

■ What is a Bootloader?

What it is: A small program that runs first on every power-on, before your application. The factory ROM bootloader (at 0x1FFF0000) can flash via UART using ST's protocol. Your custom bootloader (at 0x08000000) can receive new firmware through your own MQTT+UART pipeline.

Why it's here: Your bootloader is the ONLY firmware that never gets OTA'd. If you OTA it and it fails, the device bricks permanently. Keep it small (<28 KB), simple, and tested.

■ What is a Linker Script?

What it is: A file that tells the linker where to place each section of your binary in memory. You need TWO: one for the bootloader (ORIGIN=0x08000000) and one for the application (ORIGIN=0x08010000). Getting ORIGIN wrong causes a HardFault on the first instruction after the bootloader jumps.

Why it's here: The application must also set SCB->VTOR = 0x08010000 to relocate its interrupt vector table — without this all interrupts call the bootloader's handlers.

STM32F411 Flash Sector Layout

Sector	Address	Size	Allocated Use
--------	---------	------	---------------

0	0x08000000	16 KB	BOOTLOADER — never overwritten by OTA
1	0x08004000	16 KB	Bootloader overflow
2	0x08008000	16 KB	Config/OTA flags: pending flag + app CRC32
3	0x0800C000	16 KB	Reserved
4	0x08010000	64 KB	APPLICATION start — vector table here
5–7	0x08020000	128 KB each	Application continued (total app: 448 KB)

Two Linker Scripts

```
/* bootloader.ld */
MEMORY { FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 32K
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K }

/* application.ld */
MEMORY { FLASH (rx) : ORIGIN = 0x08010000, LENGTH = 448K
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K }

/* + in SystemInit(): SCB->VTOR = 0x08010000; */
```

Application Jump Sequence (all 7 steps are mandatory):

```
void bootloader_jump_to_app(uint32_t app_addr) {
    __disable_irq(); // 1. mask all interrupts
    SysTick->CTRL = 0; // 2. stop SysTick
    SysTick->LOAD = 0; SysTick->VAL = 0;
    SCB->VTOR = app_addr; // 3. relocate vector table
    __DSB(); __ISB(); // 4. flush CPU pipeline
    __set_MSP((volatile uint32_t *)app_addr); // 5. load app stack pointer
    void (*reset)(void) = (void(*) (void))( (volatile uint32_t *) (app_addr+4) );
    reset(); // 7. jump — bootloader no longer exists
}
```

■ What is OTA?

What it is: Over-The-Air update: deliver new firmware to the device without physical access. Your pipeline: publish .bin to MQTT → ESP32 receives in 128-byte chunks → forwards via UART protocol to STM32 → bootloader writes to flash sector by sector → CRC verifies full image → sets OTA flag → reboots into new firmware.

Why it's here: This is the payoff of everything built in Phases 1–3. The UART driver, ring buffer, FreeRTOS tasks, inter-MCU protocol, and flash driver all come together in this single pipeline.

OTA Safety Rules (non-negotiable):

- OTA 'valid' flag written LAST — only after full CRC verification of the complete image.
- Power loss during OTA must result in bootloader refusing to jump (no valid flag = no jump).
- Check received_address >= 0x08010000 before every flash write — never write to bootloader sectors.
- CRC mismatch on received image: report OTA_STATUS=CRC_FAIL, abort without touching flash.

■ Resources for this Phase

ESP-IDF OTA API Reference

docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html

esp_ota_begin/write/end API and partition table setup.

STM32 AN2606 — System Memory Boot Mode

[st.com](#) → [search AN2606](#)

Describes the factory ROM bootloader. Useful context for understanding what your custom bootloader replaces.

Quantitative Exit Gate — Phase 4

All criteria below must be met before advancing to the next phase.

- Bootloader binary size: arm-none-eabi-size confirms firmware.elf text+data < 28,000 bytes
- Bootloader jumps to v1 app (1Hz LED): jump_to_app() sequence confirmed working
- Corrupted app CRC in config sector: bootloader refuses to jump — waits for OTA
- Power cycle during OTA transfer: bootloader detects incomplete image, refuses to jump
- python ota_push.py firmware_v2.bin: STM32 reboots, LED changes to new rate without physical access
- OTA CRC verification: push a deliberately corrupted binary — STM32 reports CRC_FAIL, does not flash
- Complete OTA pipeline works twice consecutively (v1→v2, v2→v1) with zero physical access

Tags: v0.4-bootloader · v0.4-ota-pipeline · YouTube: *"Writing an STM32 Bootloader From Scratch"* · *"End-to-End OTA Over MQTT"*

PHASE 5 System Integration and Hardening

~4 weeks

Entry Gate All subsystems individually verified.

Exit Gate System runs 72 hours continuously without intervention. OTA performed mid-soak. Watchdog enabled and proven. All failure modes handled.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
System unstable only under full load (not reproducible in isolation)	High	5 days	Enable all debug logging for 24 hours. Check SystemHealth MQTT: heap trending down = memory leak. Rising sensor drop_count = queue starvation. UART error count rising = ISR race condition.
Watchdog resets the device unexpectedly	Medium	3 days	Log reset reason (RCC_CSR) in bootloader before clearing it. If SystemHealth is the cause, extend IWDG timeout to 8 seconds temporarily. Use vTaskList() to find which task is not running.
MQTT reconnection causes brief sensor data gap > 5 minutes	Medium	2 days	Check reconnection backoff cap (should be 60s max). Verify health topic reports reconnect count. If gap persists, store readings in a local ring buffer during disconnect.

■ Phase Journey — How This Phase Fits Together

Integration is where things that worked in isolation fail together. Stack sizes adequate for one task become too small under full load. The 72-hour soak is not optional — bugs that only appear after 10 hours are real.

Dependency chain: Full system bringup → Fix integration conflicts → Enable IWDG → 72-hour soak → OTA mid-soak

5.1 — Integration Bringup

When you connect all subsystems for the first time, expect at least two of these. They are normal:

- Stack overflow in a task that was fine in isolation — call depth increases under full load. Fix: increase stack in 64-word increments until vApplicationStackOverflowHook stops.
- I2C lockup after several hours — the STM32F4 errata striking under sustained operation. Fix: call bus recovery before every i2c_transfer() in SensorAcquisition.
- MQTT reconnection blocking the ESP32 main task — reconnect must be fully async in event loop.

5.2 — Hardware Watchdog (IWDG)

■ What is a Hardware Watchdog Timer?

What it is: A timer that reboots the MCU if software does not reset it within a set window. Your software kicks it every 500ms. If any task hangs, the kick stops, the timer expires, and the MCU reboots automatically. The IWDG runs on the internal LSI oscillator — it survives even a PLL crash.

Why it's here: Configure before vTaskStartScheduler(). SystemHealth task kicks it. If SystemHealth stops running, the device reboots — that is the intended behaviour.

```
// ~4 second timeout: LSI ~32kHz, prescaler /64 → ~500Hz, reload 2047
IWDG->KR = 0x5555; // unlock
IWDG->PR = 0x04; // prescaler /64
IWDG->RLR = 0x7FF; // reload = 2047
IWDG->KR = 0xCCCC; // start – cannot be stopped after this
// In SystemHealth task every 500ms:
IWDG->KR = 0xAAAA; // kick
```

5.3 — 72-Hour Soak Test Protocol

1. Flash v1.0. Verify sensor data on MQTT dashboard. Note start timestamp.
2. Hour 4: OTA update to v1.1. Verify reboot and new firmware.
3. Hour 12: cut WiFi for 30 minutes. Verify ESP32 reconnects automatically.
4. Hour 24: power cycle STM32. Verify it boots into application.
5. Hour 48: power cycle ESP32. Verify MQTT reconnects without STM32 intervention.
6. Hour 72: health topic shows uptime ~72hr, watchdog_resets = 0.

■ Resources for this Phase

MQTT Explorer

mqttxplorer.com

Dashboard view of health topic during soak test. Shows uptime and heap trending.

Quantitative Exit Gate — Phase 5

All criteria below must be met before advancing to the next phase.

- IWDG configured: freezing SystemHealth task causes MCU reboot within 4 seconds
- System health MQTT topic publishes every 10 seconds: uptime, heap, watchdog_resets visible
- 72-hour soak test: health topic shows uptime ≥ 72 hours, watchdog_resets = 0
- OTA update performed at hour 4 of soak: reboots into new firmware, soak continues
- WiFi cut for 30 minutes at hour 12: ESP32 reconnects automatically, soak continues
- Power cycle STM32 at hour 24: boots into application without bootloader waiting for OTA
- Zero unhandled HardFaults in UART log across the full 72-hour period

Tag: v1.0-system-integration · YouTube: *"72-Hour Soak Test — What Actually Failed"*

PHASE 6 PCB Design and Fabrication

~6 weeks

Entry Gate	v1.0 system integration complete. Every component's electrical behaviour understood.
Exit Gate	Fabricated PCB with all subsystems running. OTA verified on custom hardware. 24-hour power-on soak passed.

Known Hazards & Circuit Breakers

Hazard	Likelihood	Max Time-Box	Pivot if Exceeded
ERC errors from floating pins or missing power symbols	Medium	1 day	Add PWR_FLAG symbols to VCC and GND nets. Connect all unused pins explicitly to GND or VCC via 10kΩ. Use KiCad's 'no connect' X marker for intentionally unconnected pins.
DRC errors from trace too close to pad or via	Medium	2 days	Run DRC after routing each section, not only at the end. Standard JLC PCB rules: min trace 0.127mm, min clearance 0.127mm. Set these in Board Setup before routing.
PCB bringup: STM32 not detected by ST-Link	High	2 days	Check: 3.3V present at all VDD pins (measure with multimeter). Check: NRST capacitor (100nF) present. Check: SWD header wiring (SWDIO/SWDCLK not swapped). Check: BOOT0 pulled LOW (10kΩ to GND).

■ Phase Journey — How This Phase Fits Together

Phase 6 translates everything learned on dev boards into a permanent artifact. The advantage you now have: you know exactly what every component needs electrically because you debugged it live.

Dependency chain: Schematic in KiCad → Footprints → PCB layout → DRC + Gerbers → Order JLCPCB/PCBWay → Bringup + soak

■ Schematic vs PCB Layout

What it is: Schematic: a logical diagram showing which pins connect to what — no physical dimensions. PCB layout: the physical realisation — where components sit, how traces route. You must finish the schematic before starting layout.

Why it's here: Your schematic is a formalisation of the wiring you already know works. Every connection you made on dev boards in Phases 1–4 is a line in the schematic.

Learn it: KiCad Documentation

docs.kicad.org

■ Decoupling Capacitors

What it is: Every time a digital circuit switches, it draws a current spike that causes a voltage drop at the VDD pin. A 100nF ceramic capacitor placed within 1mm of the VDD pin acts as a local charge reservoir — it supplies the spike instantly.

Why it's here: Rule: one 100nF ceramic on every VDD/VDDA pin within 1mm in layout. Plus one 10μF bulk per power domain. Without decoupling, digital circuits at >10MHz behave erratically.

Install KiCad:

```
flatpak install flathub org.kicad.KiCad
```


Schematic Checklist

- 100nF decoupling cap on every VDD and VDDA pin of STM32 and ESP32
- 10µF bulk cap per power domain
- 100nF NRST filter cap on NRST pin
- 10kΩ BOOT0 pull-down to GND + test point for DFU access
- 4-pin SWD header on board edge: SWDIO, SWDCLK, GND, 3V3
- ESP32 EN RC filter: 10kΩ to 3V3 + 1µF to GND
- I2C pull-ups: 4.7kΩ on SDA and SCL to 3V3
- Level shifter footprint for HC-SR04 5V echo line
- AMS1117-3.3 LDO with 10µF+100nF on input and output
- ERC passes with zero errors before proceeding to layout

PCB Layout Rules (2-layer board)

- **Ground plane on Layer 2:** Pour solid copper on the entire bottom layer, connect all GND nets via vias. Single highest-impact layout decision.
- Decoupling caps within 1mm of their VDD pins in layout.
- No right-angle traces — use 45° bends. Avoids acid traps, reduces EMI.
- SWD header on a board edge — reachable with a Tag-Connect probe.

Fabrication: JLCPCB (jlcpcb.com) or PCBWay (pcbway.com). 5 boards in 2-layer FR4 for under ■1500 including DHL express shipping. Export Gerbers from KiCad → Plot. Upload and verify the 3D preview before confirming. Order 5 — keep 4 as spares.

■ Resources for this Phase

KiCad 8.x Documentation

docs.kicad.org

Schematic Editor and PCB Editor tutorials. Start here.

Phil's Lab — KiCad STM32 PCB Design

youtube.com/@PhilsLab

Professional PCB tutorials using KiCad. STM32 schematic videos are directly applicable.

Quantitative Exit Gate — Phase 6

All criteria below must be met before advancing to the next phase.

- ERC completes with zero errors (all nets connected, no floating pins)
- DRC completes with zero errors against JLCPCB/PCBWay 2-layer design rules
- Gerber 3D preview on JLCPCB/PCBWay matches intended component placement
- Fabricated PCB: 3.3V present at all VDD pins (measured with multimeter before first power-on)
- ST-Link detects STM32 on custom PCB — OpenOCD reports chip ID correctly
- Phase 1 driver test suite (all sensors) passes on custom PCB
- OTA update performed on custom PCB: firmware updated wirelessly, zero physical access
- 24-hour power-on soak on PCB: zero unexpected resets, health MQTT updating throughout

Tag: v1.1-pcb-bringup · YouTube: *"Designing My First IoT PCB in KiCad"* · *"PCB Arrived — First Bringup and What Failed"*

At the end of this plan: a GitHub repository with 9 tagged milestones, a documented protocol specification, a verified bootloader, an end-to-end OTA pipeline, a KiCad PCB project, and a fabricated board running your firmware — updated over WiFi. No physical access required.